

MAS Mentor Platform — Technical Guide (for Developers)

Scope: Architecture, data models, and APIs for the **MAS mentor platform** — the student-facing learning dashboard (roadmap, badges, XP) and the full admin panel (courses, modules, course-builder, batches, leads/CRM, Mr Test, Mr Learn, mapping).

Document version: 1.0 · Last updated: 2026-06-30 **Codebase:** `mr-mentor-frontend/` (Next.js admin/mentor app) + `mr-mentor-backend/` (Express + TypeORM API) **Companion doc:** `MAS_PLATFORM_SALES_GUIDE.md` (non-technical overview)

1. Architecture Orientation

Two frontends, one backend:

- mr-mentor-frontend** — the **admin + mentor** app (Next.js 15, App Router). Houses all authoring/management UI under `src/app/admin/*`, plus a mentor leaderboard and user profile.
- Student learning portal** — the student-facing dashboard (badges, XP/level/streak ribbon, batch roadmap with progress) is **served by the backend** under `/api/student/*` but **consumed by a separate portal** (`MAS_WEBSITE_URL = myanalyticsschool.com`). The backend is multi-platform via an `x-platform` header. So for the student dashboard, the **source of truth in these repos is the backend response shapes**, not React components in `mr-mentor-frontend`.
- mr-mentor-backend** — Express + TypeORM over PostgreSQL. All admin routes gated by `authMiddleware + adminMiddleware`; the frontend injects the backend JWT via `session.backendToken`.

```
flowchart TB
    ADMIN["ADMIN[mr-mentor-frontend\nadmin + mentor UI]"] -->|JWT| API["API[mr-mentor-backend\nExpress + TypeORM]"]
    PORTAL["PORTAL[Student portal\nmyanalyticsschool.com]"] -->|"/api/student/* + x-platform"| API
    API --> PG["PG[(PostgreSQL\nschemas: default, mas_crm, mrtest, mrlearn)]"]
    API -->|integration| EZ["EZ[EzExam\nexam engine = Mr Test]"]
    API -->|integration| GR["GR[Graphy\nLMS = Mr Learn]"]
    API -->|AI calls| EL["EL[ElevenLabs\nAarya voice agent]"]
    API -->|e-sign| LG["LG[Leegality\nMAS101 MOU]"]
```

Key architectural fact: Mr Test and Mr Learn are **integration consoles**, not home-grown engines. Mr Test proxies/syncs **EzExam**; Mr Learn proxies/syncs **Graphy**. The Sales CRM is fully native.

2. Core Domain Model & Mapping

The central relational flow (entity names in `mr-mentor-backend/src/entities/`):

```
Lead / RawLead / CampaignLead / VendorLead    (CRM capture)
  → User                                     (account)
  → Application (userId + batchId + courseId + status)
      status: draft → submitted → under_review → approved
      → payment_received → paid → batch_allocated → enrolled → deboarded
Batch (code unique, courseId default, assignedModuleIds[], roadmapNotes, batchLeadId/cmId/superMentorId)
  → Course (default; per-student override via Application.courseId)
      → Module[] (course_modules.courseId) → Class / Quiz / Assignment
      → roadmapItems[] (JSON on course; scheduleUnit, customTypes)
Batch × roadmapItem:
  → BatchMrLearnCourse (batchId, roadmapItemId, mrLearnCourseId, sequence, courseUrl)
  → BatchMrTestExam (batchId, roadmapItemId, mrTestExamId, sequence, prerequisiteCourseId, prerequisiteThreshold)
  → roadmapNotes[itemId] = RoadmapNote[] (per-batch session notes)
Enrollment (userId + courseId, progress, moduleProgress) ← classic LMS progress
StudentProgress / XpEvent / StudentBadge          ← gamification
```

A student "belongs to" a batch when they have an `Application` with `batchId` set and status in `{ENROLLED, BATCH_ALLOCATED, PAID}` (this exact set is used by `BadgeService.resolveEnrolledBatchIds` and `getBatchRoadmap`).

Where mapping happens in the UI: | Mapping | Screen | Service / endpoint | |-----|-----|-----| | Lead → student/application |
`admin/mas/sales/*` (CRM) | `sales.controller.ts` | | Student → batch + per-student course | `admin/mas/batches/[id]/page.tsx` |
`assignCourseToApplication` | | Batch → modules | `admin/mas/batches`, `course-builder` | `assignModulesToBatch` → `POST`
`/api/admin/mas/batches/:id/modules` | | Course → roadmap items | `admin/mas/courses` | `RoadmapSection` | | Roadmap item → Mr Learn courses /
Mr Test exams | `admin/mas/course-builder/page.tsx` | `setBatchMrLearnCourses`, `setBatchMrTestExams` |

Cross-schema references (Mr Learn `mrlearn.*`, Mr Test EzExam ids) are **string ids with no DB-level FK**; URLs are denormalized at attach time.

3. Roadmap

Where it lives: JSON on the `Course` entity (`src/entities/Course.ts`): - `roadmapItems?: RoadmapItem[]` (json) - `roadmapScheduleUnit?: ScheduleUnit` (default `'week'`) - `roadmapCustomTypes?: CustomRoadmapType[]` (json)

Shared types: backend `src/types/roadmap.ts`, mirrored in frontend `src/app/admin/mas/courses/components/types.ts`.

```
RoadmapItem = { id, order, weekNumber, type, title,
  description?, thumbnailUrl?, thumbnailS3Key?, links?[], attachments?[] }
ScheduleUnit = "week" | "day" | "session" | "step"
```

Built-in item types (`roadmapConstants.ts`): `MODULE` 📁, `TEST` 📝, `ASSIGNMENT` 🖋️, `LIVE_CLASS` 🎥, `WEBINAR` 🗣️, `PROJECT` 🚀, `MILESTONE` 🏁.

Custom types (label+emoji+color) persist in `localStorage` (`mas.roadmap.customTypes.v1`) via `customTypesStore.ts` and on the course (`roadmapCustomTypes`).

Authoring (`RoadmapSection.tsx` + `RoadmapItemCard.tsx`): - Ordering via `@dnd-kit` (`handleDragEnd` → `arrayMove` → re-stamp `order`); grouping by `weekNumber`; new item gets `weekNumber = max + 1`. - Thumbnails/attachments staged locally (`_pendingThumbnailFile`, `_pendingFile`), uploaded to S3 only at course save (`uploadService.ts` → `POST /api/admin/mas/courses/:id/roadmap/upload`, `POST /api/admin/mas/batches/:id/roadmap/:itemId/notes/upload`); S3→CloudFront via `toCdnUrl`. - **JSON import** (`JsonImportModal.tsx`): single `{course, roadmap[]}` or bulk array/ `{courses:[]}` (max 50); `parseAndValidate` requires title + positive integer weekNumber; unknown type → warning, missing → `MILESTONE`.

Batch overlay: `Batch.roadmapNotes` (json keyed by item id), plus `BatchMrLearnCourse` / `BatchMrTestExam` rows per `roadmapItemId`. Course defines the skeleton; the batch enriches each item with real content + notes.

Student view: `GET /api/student/batches/:batchId/roadmap` (`student.controller.getBatchRoadmap`, ~line 2634, enrolled-only). Resolves the student's own course (`Application.courseId` overrides `Batch.courseId`) and returns `{ batchId, courseId, scheduleUnit, items, customTypes, notes, batch, mrLearnByItem, mrTestByItem }` where the `*ByItem` maps carry per-student progress, exam unlock (prerequisite gating), and submission state (`buildMrLearnRoadmapForStudent` / `buildMrTestRoadmapForStudent`).

4. Gamification — XP, Levels, Streaks, Badges

Badges are statically defined (code, not a DB table) in `src/config/badges.config.ts` — 6 starter badges:

id	Name	Criterion	XP
<code>first_step</code>	Explorer	First <code>daily_login</code> XP event	25
<code>profile_pro</code>	All Star	<code>User.isProfileComplete</code>	25
<code>week_warrior</code>	Momentum	<code>StudentProgress.longestStreak ≥ 7</code>	50
<code>test_taker</code>	Achiever	First passing (≥50%) Mr Test on a batch-attached exam	50
<code>mentor_connector</code>	Collaborator	First <code>COMPLETED</code> mentor slot	50
<code>module_master</code>	Scholar	First completed Mr Learn module in a batch-attached course	50

Mechanics: - `StudentBadge` (`src/entities/StudentBadge.ts`) — one row per (userId, badgeId); unique `UQ_STUDENT_BADGE` makes unlock idempotent; `seenAt` gates the celebration modal. - `BadgeService` (`src/services/BadgeService.ts`) — `listBadgesForUser`, `listUnseenBadgeIds`, `markBadgesSeen`, `unlockBadge` (idempotent insert + `StudentProgressService.grantXp('badge_unlocked', baseOverride = xpReward)`), `evaluateForUser`, per-badge `evalXxx`. Batch-scoping: `resolveEnrolledBatchIds`, `attachedMrTestExamIds`, `attachedMrLearnCourseIds`.

Evaluation triggers: login (`auth.controller.ts` ~line 44), mentor-call completion (`SlotCompletionService.ts`), and the batch cron `src/workers/badgeEvaluation.worker.ts` (paginates ENROLLED/BATCH_ALLOCATED/PAID users, page size 200).

XP/level/streak: `src/config/xpRules.ts` (event base XP, action milestones, multipliers, 5-level table L1=0 → L5=2000, streak milestone every 7 days). `StudentProgress` (cached snapshot: totalXp, level, currentStreak, longestStreak) + `XpEvent` (append-only ledger, dedup (`userId,type,sourceId`)). Migration: `src/migrations/1762470000000-CreateXpAndBadges.ts`.

Student API (`studentProgress.controller.ts`, routes `student.routes.ts` 47–50): `GET /api/student/me/badges`, `GET /api/student/me/progress` (includes `unseenBadges`, `blockingAxis`, `isOnFireToday`), `POST /api/student/me/badges/seen`.

Disambiguation: `@/components/ui/badge` (shadcn) and `PredictedInterestBadge.tsx` (CRM) and `NewCourse.metadata.badges` (marketing labels) are **not** gamification badges.

5. Modules & Courses

Three distinct "course" concepts:

Concept	Entity / table	Purpose	Admin UI
Classic Course	<code>Course</code> / <code>courses</code>	Roadmap-bearing program students follow	<code>admin/mas/courses/page.tsx</code>
New Course (MasCourse)	<code>NewCourse</code> / <code>mas_courses</code>	Marketing catalog for public site (type, slug, metadata, pricing tiers)	<code>admin/mas/new-courses/page.tsx</code>
Course Plan	<code>CoursePlan</code> + <code>CoursePlanLecture</code>	AI-generated multi-lecture courses → AI Classrooms	<code>admin/mas/course-plans/page.tsx</code>

Module (`src/entities/Module.ts`, table `course_modules`): child of a course (`courseId` FK CASCADE), `title`, `moduleOrder`, owns `Class` / `Quiz` / `Assignment`, counters `totalClasses/Quizzes/Assignments`. A roadmap item of `type:"MODULE"` is a *display reference*, **not** the same record as a `course_modules` row. - Admin: `admin/mas/modules/page.tsx` (create/edit/delete/duplicate/reorder) → `adminMasModuleService` → `/api/admin/mas/modules*` (admin.routes 361–397). Also editable in **Course Builder** with drag-reorder. - **Module** → **batch**: `Batch.assignedModuleIds[]` set via `assignModulesToBatch` → `POST /api/admin/mas/batches/:id/modules`.

Course Builder (`admin/mas/course-builder/page.tsx`, ~3300 lines) — the integration workbench: manage a course's modules, assign modules to a batch, and **map external content to roadmap items**: - Mr Learn (Graphy) per item: `setBatchMrLearnCourses` → `POST /api/admin/mas/batches/:id/mrlearn-courses` (`[{roadmapItemId, mrLearnCourseIds[]}]`); catalog via `adminMrLearnCatalogService`. - Mr Test (EzExam) per item with prereq gate: `setBatchMrTestExams` → `POST /api/admin/mas/batches/:id/mrtest-exams`; catalog via `adminMrTestCatalogService`. - Composite dnd ids `${roadmapItemId}::${courseId}` keep reordering within a bucket.

6. Batches

Entity (`src/entities/Batch.ts`, table `batches`): `code` (unique), `name`, `courseId?`, `status` (upcoming/active/completed/cancelled), `startDate` / `endDate`, `maxStudents`, `currentStudents`, `originalPrice?`, `enrollmentFee?`, `gstEnabled` / `gstPercent`, `requiresDocuments` + `documentRequirements`, `bannerImageUrl` / `S3Key`, `paymentType`, `features[]`, `assignedModuleIds[]`, `superMentorId`, `cmId`, `batchLeadId`, `whatsappLinks`, `roadmapNotes`.

Create flow (`admin/mas/batches/page.tsx`): 1. Create modal (`resetForm`). 2. `formData`: code (auto-uppercase, required), name, courseId (optional → "Independent Batch"), dates, maxStudents (slider step 10, default 50), `enrollmentFee` (offer, required >0), `originalPrice` (MRP, ≥ offer), GST, document requirements, banner (from `bannerTemplates`), batch lead, whatsapp links. 3. Client `validate`: code required, `enrollmentFee`>0, `originalPrice`≥`enrollmentFee`. 4. `handleCreateBatch` → `adminMasBatchService.createBatch` → `POST /api/admin/mas/batches` → `adminMas.controller.createBatch` → `batchService.createBatch`. 5. Changing `courseId` on an existing batch can return `code:'REQUIRES_REMAP_CONFIRMATION'` → re-submit with confirm flag (`handleUpdateBatch` ~177–193; controller ~292).

Enrollment / mapping: via `Application` (not a direct join). `admin/mas/batches/[id]/page.tsx` lists applications and assigns a course per student → `assignCourseToApplication(applicationId, courseId)` (students in one batch may follow different courses).

Analytics: `admin/sections/BatchKPISection.tsx`, `BatchDetailView.tsx` (`GET /api/admin/dashboard/batch/:batchName/students` → per-student connects/tokens/revenue + Excel export), plus `BatchWiseSection`, `ConnectsTrendSection`, `TokensTrendSection`, `RevenueOverviewSection`.

`CreateAaryaBatchModal.tsx` is a **sales AI-call batch**, unrelated to academic batches.

7. Mr Test (EzExam integration)

Admin console for **EzExam** (`myanalyticsschool.ezexam.in`). Browse exams, push MAS students as test-takers, sync results back into MAS, attach exams to batch roadmap items (optionally gated by a Mr Learn course prerequisite).

- **Frontend** `src/app/admin/mrtest/`: `page.tsx` (live exams `GET /api/ezexam/online-exams`), `exams/[onexId]`, `students/`, `create-student/`, `synced/[configId]` (cached snapshot), `settings/` (auth + sync configs). Shared `?tag=` filter from EzExam student tags.
- **Backend**: routes `src/routes/ezexam.routes.ts` (mounted `/api/ezexam`); controllers `ezexam.controller.ts` (live pass-through) + `mrtestSync.controller.ts`; services `EzExamService.ts`, `MrTestSyncService.ts`, `MrTestTrainingDataService.ts`.
- **Entities (schema `mrtest`)**: `MrTestSyncConfig` (one per synced exam: `onlineExamId`, `batchId?`, `interval`, `enabled`, `lastRun`), `MrTestOnlineExam`, `MrTestSubmission` (marks/percentage/rank), `MrTestSubmissionReport` (question-level), `MrTestSyncRun` (audit). Cross-link: `BatchMrTestExam` (default schema) — `mrTestExamId` + `roadmapItemId` + `prerequisiteCourseId` / `prerequisiteThreshold` (default 75%) + `takeUrl`.
- **Key endpoints**: auth `POST /auth/login/logout`, `GET /auth/status`; live `GET /online-exams`, `/online-exams/:onexId/detail`, `POST /proxy`; students `GET /students`, `POST /students/create-mas-account`, `GET /students:rollnum/report`; sync `GET/POST /sync/configs`, `POST /sync/configs/:id/run`, `POST /sync/batches/:batchId/run`, `POST /sync/by-tag`.
- **"Synced"** = results pulled (cached) into the `mrtest` schema via a `MrTestSyncConfig` + background runs (vs. the live "Online exams" home).

8. Mr Learn (Graphy integration)

Admin console for **Graphy** LMS (`/api/graphy`). Browse video courses, manage learners/admins, inspect question bank, download CSV reports, push MAS students as learners, sync per-learner progress, send **WhatsApp progress reminders**.

- **Frontend** `src/app/admin/mrlearn/` : `page.tsx` (courses), `courses/[courseId]`, `students/`, `admins/`, `questions/`, `reports/`, `create-student/`, `synced/[configId]`, `settings/` (auth, sync configs, reminder settings, new-student cron).
- **Backend**: routes `graphyLms.routes.ts` (mounted `/api/graphy`); controllers `graphyLms.controller.ts` + `mrLearnSync.controller.ts` ; services `GraphyLmsService.ts`, `MrLearnSyncService.ts`, `MrLearnReminderService.ts` .
- **Entities (schema `mrLearn`)**: `MrLearnCourse`, `MrLearnLearner` (userId link, progressPercentage, completion, report jsonb), `MrLearnSyncConfig` (reminderEnabled, intervalHours default 168, WhatsApp template `mrlearn_course_progress_reminder`), `MrLearnSyncRun`, `MrLearnLearnerReport`, `MrLearnReminderLog`, `MrLearnAuthCredentials` . Cross-link: `BatchMrLearnCourse` (default schema).
- **Key endpoints**: auth `/auth/*` ; content `GET /courses`, `/courses/:id/learners`, `POST /courses/:id/export`, `GET /questions`, `/reports`, `/reports/:id/download`, `POST /proxy` ; sync `GET/POST /sync/configs`, `POST /sync/configs/:id/run`, `POST /sync/batches/:batchId/run`, `POST /sync/students/seed-all-missing` ; reminders `POST /sync/configs/:id/reminders/send`, `GET /sync/configs/:id/reminders/logs` ; cron `GET/PATCH /sync/new-student-cron` .

Mr Test = exams (EzExam); Mr Learn = video courses + progress (Graphy). Same sync architecture (configs → cached snapshot → runs); Mr Learn adds WhatsApp reminders + new-student auto-sync cron.

9. Leads / Sales CRM (native)

Native CRM: capture leads → distribute → qualify (incl. AI calls) → convert to `Application` → assign to `Batch` → MAS101 Pay-After-Placement.

Lead tables (schema `mas_crm` unless noted): - `RawLead` — primary CRM lead (source csv/vendor/website, sessionYear, course, `owningSalesHeadId`, `assignedTo`, `assignedCounsellor`, `interestLevel` hot/warm/cold, `webinarStatus`, follow-up, `callCount`, `convertedAt` / `closedAt`). Lifecycle bucket is **derived** in `RawLeadService` (new|contacted|qualified|converted|closed|untouched). - `CampaignLead` (marketing funnel), `VendorLead` (vendor API → mirrored to `RawLead`), `Lead` (default schema, legacy/application-linked). - Supporting: `LeadImportBatch`, `LeadTag`, `LeadFollowUp`, `LeadActivityLog`, `LeadCallLog`, `LeadEmailLog`, `LeadSmsLog`, `LeadWhatsAppLog`, `LeadAssignmentHistory`, `LeadAssignmentApproval` .

Frontend: sales dashboard `admin/mas/sales/page.tsx` (tabs: Overview/Applications/Leads/Discounts/Campaign); CRM `admin/mas/sales/crm/` (`CrmSidebar` buckets, `page.tsx` 50/page + `CrmRefinePanel`, `Leads/[id]`, `today`, `targets`, `analytics`, `aarya/`); `PredictedInterestBadge.tsx`; `AssignmentApprovalsButton.tsx` (sales-head approvals); `workflows/`; `mas101-post-payment/`; lead pool `admin/lead-pool/page.tsx` .

Aarya (AI calling): ElevenLabs-backed voice agent. `CreateAaryaBatchModal` → `aaryaService` ; entity `AaryaCallBatch` ; service `AaryaCallSyncService.ts` (polls ElevenLabs every 15 min, writes duration, computes `RawLead.predictedInterestLevel` via `bucketFromDuration` : ≥30s hot, ≥10s warm, >0 cold; also wakes `WorkflowEnrollment` rows waiting on a call); routes `aarya.routes.ts` ; worker `aaryaSync.worker.ts` .

Sales automation workflows (admin-only): React Flow builder `workflows/components/WorkflowBuilder.tsx` ; node types (`nodeMeta.ts`): `trigger`, `aaryaCall`, `branch` (split by Aarya interest), `sendEmail`, `sendWhatsApp`, `wait` . Backend `salesWorkflow.routes.ts` (`/api/admin/workflows` : CRUD, `/scan-now`, `/:id/run-manual`, `/:id/enrollments`), `WorkflowEnrollment.entity.ts`, worker `workflow.worker.ts` (trigger scan every 5 min). **Distinct from Mr Hire recruitment workflows — don't conflate.**

Conversion → Application → Batch: `sales.controller.ts` (`getApplications`, `updateApplicationStatus`, `assignApplication`); `PATCH /api/sales/applications/:id/batch` and `/course` map an application to a batch/course; `RawLead.convertedAt` stamped; batch-leads via `batchLead.routes.ts` (`/api/batchlead`).

MAS101 PAP (Pay-After-Placement): `Mas101PapWorkflow` entity (status `pending_mou_details` → `pending_mou_upload` → `pending_mou_review` → `mou_signed_approved` → `enrolled`, Leegality e-sign, payment plan/status, placement status) + `Mas101PapAgreementTemplate` ; controller `mas101PapWorkflow.controller.ts` ; frontend `mas/sales/mas101-post-payment/` (React Flow status board). Endpoints under `/api/sales/...` .

10. Student Dashboard API Surface (backend-served)

The student portal consumes these (`student.routes.ts` + controllers): | Area | Endpoint | |-----|-----| | Dashboard home | `GET /api/student/dashboard` (enrollments, announcements, application status, batch-lead, MAS101 state) | | Roadmap | `GET /api/student/batches/:batchId/roadmap` (enrolled-only; per-student course + Mr Learn/Mr Test state) | | Progress ribbon | `GET /api/student/me/progress` (level, XP, streak, unseenBadges, blockingAxis) | | Badges | `GET /api/student/me/badges`, `POST /api/student/me/badges/seen` | | Quizzes | `/api/student/quizzes*` | | AI Classrooms | `/api/student/ai-classrooms*` (from Course Plans) | | Job applications | `/api/student/job-applications` (Mr Hire) | | Mentor calls / meetings | mentor/slot endpoints (`mentorBookings.controller`) | | Tokens | `token.routes` (`Tokens` / `TokenUsage` / `TokenPurchase`) | | Profile | `/api/student/profile*` (`StudentProfile`, completion, resume, email-OTP) |

In `mr-mentor-frontend` the only genuinely student/user-facing pages are thin: `profile/page.tsx` (settings), `student-profile/page.tsx` (10-line stub), `leaderboard/` (mentor leaderboard, not student XP), `courses/`, `meetings/`, `token-transactions/` .

11. Backend Route Mounting (`src/routes/index.ts`)

Mount	Module
<code>/api/sales</code>	sales, emailTemplate, mas101PapWorkflow
<code>/api</code>	campaignLead, rawLead, aarya, salesWorkflow
<code>/api/batchlead</code>	batchLead
<code>/api/graphy</code>	Mr Learn (Graphy)
<code>/api/ezexam</code>	Mr Test (EzExam)
<code>/api/admin/mas/*</code>	courses, batches, modules, mrlearn-courses, mrtest-exams
<code>/api/student/*</code>	student dashboard, roadmap, progress, badges, quizzes, etc.

Background workers: `badgeEvaluation.worker.ts` , `aaryaSync.worker.ts` (15 min), `workflow.worker.ts` (5 min), lead auto-assignment (15 min), Mr Learn reminder + new-student cron.

12. Key File Index

Roadmap authoring: `mr-mentor-frontend/src/app/admin/mas/courses/components/{RoadmapSection,RoadmapItemCard,TypePicker,JsonImportModal}.tsx` , `{roadmapConstants,types,customTypesStore,uploadService}.ts` . **Batches:** `admin/mas/batches/page.tsx` , `batches/[id]/page.tsx` , `batches/components/*` , `admin/sections/{BatchKPISection,BatchDetailView}.tsx` . **Modules/courses:** `admin/mas/{modules,course-builder,course-plans,new-courses}/page.tsx` . **CRM:** `admin/mas/sales/**` , `admin/lead-pool/page.tsx` . **Service:** `src/services/adminMasService.ts` . **Mr Test FE/BE:** `admin/mrtest/**` ; `src/routes/ezexam.routes.ts` , `controllers/{ezexam,mrtestSync}.controller.ts` , `services/{EzExamService,MrTestSyncService}.ts` , `entities/mrtest/*` , `entities/BatchMrTestExam.ts` . **Mr Learn FE/BE:** `admin/mrlearn/**` ; `routes/graphyLms.routes.ts` , `controllers/{graphyLms,mrlearnSync}.controller.ts` , `services/{GraphyLmsService,MrLearnSyncService,MrLearnReminderService}.ts` , `entities/mrlearn/*` , `entities/BatchMrLearnCourse.ts` . **Gamification:** `src/config/{badges.config,xpRules}.ts` , `entities/{StudentBadge,StudentProgress,XpEvent}.ts` , `services/{BadgeService,StudentProgressService}.ts` , `workers/badgeEvaluation.worker.ts` , `migrations/1762470000000-CreateXpAndBadges.ts` . **Core entities:** `src/entities/{Course,Batch,Module,Enrollment,Application,RawLead,CampaignLead,VendorLead,Lead,AaryaCallBatch,Mas101PapWorkflow,NewCo`

Canonical design narrative: `mr-mentor-backend/ROADMAP_COURSE_BATCH_TEST_CONNECTIONS.md` .

Security: EzExam/Graphy/Leegality/ElevenLabs credentials live in env / `MrLearnAuthCredentials` / `MrTestSyncConfig` — never commit them; rotate anything exposed in plaintext. (Reminder: `mr-mentor-backend/.env` line 34 looked like a live secret.)